

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Программирование графических процессоров»

Освоение программного обеспечения для работы с технологией CUDA.

Примитивные операции над векторами.

Выполнил: *Серый Никита Олегович*

Группа: *M8O-409B-22*

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

1. Цель работы: Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений(CUDA).
Реализация одной из примитивных операций над векторами.
2. Вариант 7. Поэлементное вычисление модуля вектора.

Входные данные. На первой строке задано число n -- размер векторов. На следующей строке записано n вещественных чисел -- элементы вектора.

Выходные данные. Необходимо вывести n чисел -- результат поэлементного вычисления модуля исходного вектора.

Программное и аппаратное обеспечение

Графический процессор (Google Colab)

Compute capability	7.5
Name	Tesla T4
Total Global Memory	15828320256
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	6553
Multiprocessors count	40

Процессор AMD Ryzen 5 5500U (2.1 GHz, max 4 GHz)

Technology	7 нм
Cores	6
Threads	12
Cache (Level 2)	64К (на ядро)
Cache (Level 2)	512К (на ядро)
Cache (Level 3)	8 МБ (всего)

Оперативная память

Number of modules	2
Module Size	8 ГБ
Frequency	3200 МГц
Generation	DDR4

Жесткий диск - 512 ГБ SSD

OS - Ubuntu 22

IDE - VS Code

Compiler - nvcc

Метод решения

В основе решения **распараллеливание данных**. Вместо последовательного обхода массива в цикле на CPU, задача распределяется между тысячами мелких потоков GPU.

Программа разделена на два контекста выполнения:

1. **Host (CPU)**: Последовательно управляет логикой программы, выделяет память в ОЗУ, инициализирует данные и координирует запуск вычислений на ускорителе.
2. **Device (GPU)**: Выполняет тяжелые вычислительные задачи в массивно-параллельном режиме.

Описание программы

Программа построена по схеме управления ресурсами в гетерогенных вычислениях:

1. **Host**: считывает размер массива n и сами данные из стандартного ввода в оперативную память.
2. **Память**: `cudaMalloc` выделяет буфер в глобальной памяти видеокарты, данные передаются командой `cudaMemcpy`.
3. **Kernel**: Запускается функция `kernel` на видеокарте. Конфигурация `<<<1024, 1024>>>` означает запуск 1024 блоков по 1024 потока в каждом (всего более миллиона потоков одновременно).
4. **Синхронизация и возврат**: После завершения работы ядра результаты копируются обратно, и память освобождается.

Особенности реализации

Для обеспечения надежности используется макрос `CSC (CUDA Safe Call)`. Он оборачивает каждый вызов CUDA API, проверяет возвращаемое значение `cudaError_t` и, в случае сбоя, выводит сообщение об ошибке с указанием файла и строки.

Форматированный вывод позволяет выводить результаты с высокой точностью.

Ядро:

```
__global__ void kernel(float *arr, int n) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    int offset = blockDim.x * gridDim.x;  
    while(idx < n) {  
        arr[idx] = abs(arr[idx]);  
        idx += offset;  
    }  
}
```

В каждом потоке вычисляется уникальный индекс. Внутри цикла каждый поток считывает значение из глобальной памяти, применяет функцию модуля и записывает результат обратно, обеспечивая высокую пропускную способность за счет объединения запросов к памяти. Если размер массива n больше, чем общее количество запущенных потоков, потоки перемещаются по массиву с шагом, равным размеру сетки (offset).

Результаты

1. Отобразить в виде таблички или графиков замеры времени работы ядер с различными конфигурациями (начиная с <<< 1, 32 >>> и как минимум до <<< 1024, 1024 >>>). **Обязательно указать единицы измерения времени.**
2. Произвести сравнение с CPU (для этого нужно реализовать свой вариант ЛР без использования технологии CUDA / OpenMP).

Замеры времени работы ядер с различными конфигурациями, а также замер работы аналогичной программы без использования технологии CUDA - на CPU.

1. Конфигурация <<< 1, 32 >>>

Execution time: 0.094208 ms

2. Конфигурация <<< 128, 64 >>>

Execution time: 0.097024 ms

3. Конфигурация <<< 512, 512 >>>

Execution time: 0.098304 ms

4. Конфигурация <<< 1024, 1024 >>>

Execution time: 0.144512 ms

5. CPU

Chrono: 0.120896 ms

```
%%writefile cpu_compare.cpp
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>
#include <iomanip>

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    std::vector<float> arr(n);
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }
    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < n; ++i) {
        arr[i] = std::abs(arr[i]);
    }

    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<float, std::milli> duration = end - start;

    for (int i = 0; i < n; ++i) {
        std::cout << std::scientific << std::setprecision(10) << arr[i] << " ";
    }

    std::cout << "\nCPU Execution time: " << duration.count() << " ms" << std::endl;

    return 0;
}
```

... Writing cpu_compare.cpp

Рис. 1 - Вариант ЛР без использования технологии CUDA

Выводы

В ходе выполнения лабораторной работы я ознакомился с программно-аппаратной архитектурой параллельных вычислений (CUDA), а также реализовал одну из примитивных операций над вектором - поэлементное вычисление его модуля.

Этот алгоритм может применяться в области компьютерного зрения и обработки изображений (слияние изображений, фильтрация изображений), в области машинного обучения и анализа данных (агрегация данных, обучение нейросетей), в области финансового анализа (оценка рисков, оптимизация портфеля) и т.д.

Программировать было несложно, поскольку вариант работы оказался проще, чем пример с лекции. Основная сложность:

1) Ошибка форматирования. Решил это, указав верный спецификатор формата без необходимости замены типа с `int`.

Сравнивая время работы ядер с различными конфигурациями и размерами массивов, а также аналогичной программы без использования технологии CUDA - на CPU, можно заметить, что лучше всего с применением технологии CUDA справляется конфигурация <<< 1, 32 >>> в силу работы на маленьком тесте.

Выполнение тестов на CPU показало, что алгоритм работает так быстрее, чем на конфигурации <<<1024, 1024 >>>. При правильном выборе конфигурации допускается значительное увеличение производительности алгоритма на больших данных.